

Projet Optimisation Finances

Javier Peña Colada, Martin Leivasion Fiscale, et Autiste Pras

1 Modélisation, pseudo code et résolution “à la main”

1. Soit $G = (X, U)$ un graphe orienté, où E = Euros, D = Dollars, F = Francs Suisses, J = Yens, on a :

$$X = \{E, D, F, J\}$$

$$U = \{(E, D), (D, E), (J, D), (D, J), (F, D), (D, F), (E, F), (F, E), (E, J), (J, E), (J, F), (F, J)\}$$

$$\begin{array}{ll} v(E, D) = 1,19 & v(D, E) = 0,84 \\ v(D, J) = 1,12 & v(J, D) = 0,89 \\ v(D, F) = 1,37 & v(F, D) = 0,73 \\ v(E, F) = 1,62 & v(F, E) = 0,62 \\ v(E, J) = 1,33 & v(J, E) = 0,75 \\ v(J, F) = 1,22 & v(F, J) = 0,82 \end{array}$$

2.

$$v(\mu) = \prod_{i=0}^{n-1} v(x_i, x_{i+1})$$

où x_i, x_{i+1} sont les sommets du graphe et où $v(x_i, x_{i+1})$ désigne la valeur de l'arête reliant x_i à x_{i+1} (c'est-à-dire le taux de change entre ces deux monnaies). Le produit des $v(x_i, x_{i+1})$ représente les échanges successifs réalisés.

3. En partant d'un montant en euros, on cherche à effectuer des échanges successifs vers d'autres monnaies afin d'augmenter le montant final obtenu en euros.

On cherche donc à trouver un cycle $\mu = (x_0, x_1, \dots, x_n)$, où $x_0 = x_n = E$ et tel que $v(\mu)$ soit maximal, c'est à dire tel que $v(\mu) = \prod_{i=0}^{n-1} v(x_i, x_{i+1})$ soit le plus grand possible.

Si $v(\mu) > 1$, le cycle est profitable : plus $v(\mu)$ est grand, plus le profit est important.

Si $v(\mu) < 1$, le cycle est défavorable : plus $v(\mu)$ est petit, plus la perte est importante.

Si $v(\mu) = 1$, le cycle est neutre, sans gain ni perte.

4. Si le graphe G contient un cycle fermé μ tel que $v(\mu) > 1$, alors en répétant ce cycle une infinité de fois, on peut espérer un profit infini.
5. Voir les pseudo-codes sur la page 2.
6. La meilleure séquence d'échange est **{E, D, F, E}** et offre un taux multiplicatif de **1.010786**. C'est-à-dire qu'en investissant 100€, on aurait après ces 3 échanges 101,0786€.

Algorithm 1 meilleur_cycle($G, debut, p$)

```
1: Entrées : matrice d'adjacence  $G$  des taux  $G[i, j]$ , monnaie de départ  $debut$ , nombre max  
   d'échanges  $p$   
2: Sorties : ( $meilleur\_gain, meilleur\_chemin$ ) revenant à  $debut$  avec  $\leq p$  échanges  
3:  $meilleur\_gain \leftarrow 1.0$ ;  $meilleur\_chemin \leftarrow$  liste vide  
4: procedure DFS( $noeud, gain, chemin, restants$ )  
5:   if  $restants = 0$  then  
6:     if  $noeud = debut$  and  $gain > meilleur\_gain$  then  
7:        $meilleur\_gain \leftarrow gain$ ;  $meilleur\_chemin \leftarrow chemin$   
8:     return ▷ On explore tous les voisins possibles (on peut revisiter des nœuds)  
9:   for chaque  $v$  où  $G[noeud, v] > 0$  do  
10:     $gain' \leftarrow gain \times G[noeud, v]$   
11:     $chemin' \leftarrow chemin \cup \{(noeud \rightarrow v)\}$   
12:    if  $v = debut$  and  $gain' > meilleur\_gain$  then  
13:       $meilleur\_gain \leftarrow gain'$ ;  $meilleur\_chemin \leftarrow chemin'$  ▷ On enregistre tout cycle  
qui revient à  $debut$   
14:    DFS( $v, gain', chemin', restants - 1$ )  
15: DFS( $debut, 1.0, [], p$ )  
16: return ( $meilleur\_gain, meilleur\_chemin$ )
```

Algorithm 2 meilleur_cycle_ameliore($G, debut, p$)

```
1: Init :  
2: for  $k \leftarrow 0$  à  $p$  do  
3:   for chaque  $u$  dans sommets( $G$ ) do  
4:      $dp[k][u] \leftarrow 0.0$ ;  $parent[k][u] \leftarrow \text{NULL}$   
5:  $dp[0][debut] \leftarrow 1.0$   
6: Relaxations :  
7: for  $k \leftarrow 1$  à  $p$  do  
8:    $updated \leftarrow \text{FAUX}$   
9:   for chaque  $u$  dans sommets( $G$ ) do  
10:    if  $dp[k-1][u] \leq 0.0$  then  
11:      continue  
12:     $base \leftarrow dp[k-1][u]$   
13:    for chaque  $(v, rate)$  dans  $G[u]$  do  
14:       $alt \leftarrow base \times rate$   
15:      if  $alt > dp[k][v]$  then  
16:         $dp[k][v] \leftarrow alt$ ;  $parent[k][v] \leftarrow u$ ;  $updated \leftarrow \text{VRAI}$   
17:   if  $updated = \text{FAUX}$  then  
18:     break ▷ aucun progrès, on stoppe tôt  
19: Choix du meilleur retour sur  $debut$  :  
20:  $best\_gain \leftarrow 0.0$ ;  $best\_k \leftarrow \text{NULL}$   
21: for  $k \leftarrow 1$  à  $p$  do  
22:   if  $dp[k][debut] > best\_gain$  then  
23:      $best\_gain \leftarrow dp[k][debut]$ ;  $best\_k \leftarrow k$   
24: if  $best\_k = \text{NULL}$  then  
25:   return ( $0.0, \text{liste vide}$ ) ▷ aucun cycle trouvé  
26: Reconstruction du cycle :  
27:  $cycle\_inv \leftarrow$  liste vide;  $cur \leftarrow debut$   
28: for  $step \leftarrow best\_k$  à  $1$  par pas  $-1$  do  
29:    $prev \leftarrow parent[step][cur]$   
30:   if  $prev = \text{NULL}$  then  
31:     return ( $best\_gain, \text{liste vide}$ ) ▷ sécurité  
32:   append  $prev$  à  $cycle\_inv$ ;  $cur \leftarrow prev$   
33:  $cycle \leftarrow \text{reverse}(cycle\_inv)$ ; append  $debut$  à la fin de  $cycle$   
34: return ( $best\_gain, cycle$ )
```
